微博话题评论爬虫 - 代码思路说明书

整体概述

本程序用于爬取微博话题（如 `#男童被机器人踢中腹部痛苦倒地#` ）下的微博数据，并进行数据分析、可视化和生成报告。

一、环境配置与初始化

1.1 依赖库引入

```
import requests           # HTTP请求
  from bs4 import BeautifulSoup  # HTML解析
  from urllib.parse import quote # URL编码
  import csv                # CSV文件处理
  import re                 # 正则表达式
  import os                 # 文件操作
  import time               # 延时控制
  import pandas as pd       # 数据分析
  import jieba              # 中文分词
  import jieba.analyse      # 关键词提取
  from collections import Counter  # 词频统计
  import matplotlib.pyplot as plt  # 图表绘制
  from wordcloud import WordCloud  # 词云生成
  import numpy as np        # 数值计算
  from PIL import Image     # 图像处理
```

1.2 中文字体设置

```
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
  plt.rcParams['axes.unicode_minus'] = False
```

思路： matplotlib 默认不支持中文，需要指定支持中文的字体，否则图表中文会显示为方块。

1.3 读取Cookie与请求头配置

```
with open('cookie.txt', 'r') as f:
      cookie = f.read().strip()

  headers = {
      'referer': 'https://passport.weibo.com/',
      'user-agent': '...',
      'cookie': cookie
  }
```

思路：

- 微博需要登录才能访问，cookie 是登录凭证
- `referer` 设置为微博 passport 域名，模拟正常访问来源
- 使用 requests.Session() 保持会话，自动携带 cookie

二、数据爬取模块

2.1 `extract_weibo_data(html, page_num)` - 解析微博数据

思路：从微博搜索结果页面 HTML 中提取结构化数据。

1. **使用 BeautifulSoup 解析 HTML**

```
soup = BeautifulSoup(html, 'html.parser')
```

1. **查找微博卡片**
   - 优先查找 `action-type="feed_list_item"` 的 div（微博标准结构）
   - 备用查找 `class="card-wrap"` 的 div

```
cards = soup.find_all('div', attrs={'action-type': 'feed_list_item'})
  if not cards:
      cards = soup.find_all('div', class_=re.compile(r'card-wrap'))
```

1. **逐个解析每条微博**
   - **用户昵称/主页**：通过 `.name` 选择器获取
   - **头像URL**：通过 `.avator img` 获取，并补全相对路径
   - **发布时间/来源**：通过 `.from a:first-child` 和 `a:last-child` 获取
   - **微博正文**：

        - 优先取 `node-type="feed_list_content_full"`（完整内容）
        - 备用取 `node-type="feed_list_content"`（截断内容）
    - **互动数据**：通过 `action-type` 属性定位转发/评论/点赞按钮
    - **微博MID**：从卡片属性 `mid` 获取，拼装微博链接
2. **清理正文**

```
re.sub(r'#.*?#', '', text)    # 移除话题标签
  re.sub(r'http\S+', '', text) # 移除URL
  re.sub(r'@\S+', '', text)    # 移除@用户
```

1. **返回有效数据**：只有昵称或正文非空才保留

2.2 `append_to_csv(data, filename)` - 数据持久化

思路：将数据追加写入 CSV 文件，支持增量爬取。

1. 检查文件是否存在决定是否写入表头
2. 使用 `csv.DictWriter` 以字典方式写入，方便对应字段
3. `encoding='utf-8-sig'` 确保中文不乱码

2.3 `get_total_pages(html)` - 提取总页数

思路：从页面中解析搜索结果总页数。

1. 查找 `.m-page .list a` 中的页码链接，取最大值
2. 如果只有1页，尝试从错误信息中提取"找到X条结果"，计算页数 = (总数+19)//20

2.4 `crawl_weibo_pages(start_page, max_pages, output_file)` - 爬取主流程

思路：循环爬取多页微博数据。

```
URL格式: https://s.weibo.com/weibo?q=%23关键词%23&page=页码
```

流程：

1. **循环翻页**：从起始页开始，逐页爬取
2. **检测登录拦截**：如果页面包含"Sina Visitor System"或"登录"，说明 cookie 失效，停止爬取
3. **提取数据**：调用 `extract_weibo_data` 解析页面
4. **判断终止条件**：
    - 无数据时停止
    - 达到最大页数时停止
    - 被拦截时停止
5. **增量写入**：每页数据立即追加到 CSV

6. **页数推断**：第一页从响应中获取总页数，作为后续爬取上限
7. **延时控制**：每页间隔 2 秒，避免请求过快被封禁

三、数据分析模块

3.1 `text_preprocessing(text)` - 文本预处理

思路：清洗微博正文，去除干扰字符，保留纯文本用于分词分析。

处理步骤：

1. 空值处理
2. 去除URL：`http[s]?://...` 匹配
3. 去除话题标签：`#...#` 匹配
4. 去除@用户：`@[\w\u4e00-\u9fff]+` 匹配
5. 去除表情符号：`[.*?]` 匹配（微博表情如[哈哈]）
6. 去除非汉字：`[^\u4e00-\u9fa5]` 只保留中文
7. 合并空白字符

3.2 `perform_analysis(csv_file)` - 分析主流程

执行6大类分析，最后生成文本报告。

3.2.1 用户统计分析

- 统计每个用户的发帖数量
- 取 TOP10 活跃用户
- 生成柱状图保存

3.2.2 发布时间分析

- 提取发布时间字段
- 统计各时间的微博分布
- 生成柱状图

3.2.3 互动数据分析

- 转发数、评论数、点赞数转换为数值
- 计算总量和均值
- 计算"总互动 = 转发 + 评论 + 点赞"
- 取 TOP10 高互动微博
- 绘制总互动量柱状图

3.2.4 文本内容分析

1. **分词处理**：使用 `jieba.cut` 对每条正文分词
2. **过滤**：去除单字词（len >= 2），只保留有意义的词
3. **词频统计**：使用 `Counter` 统计词出现次数
4. **输出**：保存 TOP50 词频到 CSV

3.2.5 生成词云图

- 将词频字典传给 `WordCloud` 生成器
- 配置：1200x800 像素、白色背景、200最大词数、viridis配色
- 优先使用 `simhei.ttf` 字体（备用方案不指定字体）
- 保存为 PNG 图片

3.2.6 情感倾向分析

- **情感词库法**（简化版）：
  - 正面词库：好、支持、希望、安全、赔偿、负责、关心、没事、万幸
  - 负面词库：危险、问题、担心、痛心、愤怒、失望、冷漠、寒心、受伤、痛苦
- 遍历每条文本，匹配情感词
- 绘制正面/负面/中性三分类的饼图

3.2.7 生成分析报告

将所有分析结果写入 `analysis_report.txt` 文本文件，包含：

- 数据概况（总微博数、总页数）
- 用户分析（活跃用户数、TOP5）
- 互动数据（总量、均值）
- 内容分析（有效文本数、关键词TOP10）
- 情感分析（各类占比）

四、主程序入口

```
if __name__ == "__main__":
      START_PAGE = 1       # 起始页
      MAX_PAGES = 30      # 最大页数（可改为None爬全部）
      OUTPUT_FILE = 'weibo_data.csv'

      # 步骤1：爬取数据
      csv_file = crawl_weibo_pages(start_page=START_PAGE, max_pages=MAX_PAGES, output_f
  ile=OUTPUT_FILE)

      # 步骤2：数据分析
      if os.path.exists(csv_file):
          perform_analysis(csv_file)
```

流程：先爬取数据到 CSV，再读取 CSV 进行分析。爬取完成后会在当前目录生成：

- `weibo_data.csv` - 原始微博数据
- `analysis_results/` 目录 - 所有分析结果

五、输出文件清单

文件名	说明
weibo_data.csv	原始微博数据
analysis*results/user*activity_top10.csv	用户活跃度TOP10
analysis*results/user*activity_chart.png	用户活跃度柱状图
analysis*results/post*time_distribution.png	发布时间分布图
analysis*results/top*interactive_weibo.csv	高互动量微博TOP10
analysis*results/interaction*summary.png	互动量统计图
analysis*results/word*frequency.csv	词频统计
analysis_results/wordcloud.png	词云图
analysis*results/sentiment*distribution.png	情感分布饼图
analysis*results/analysis*report.txt	完整分析报告